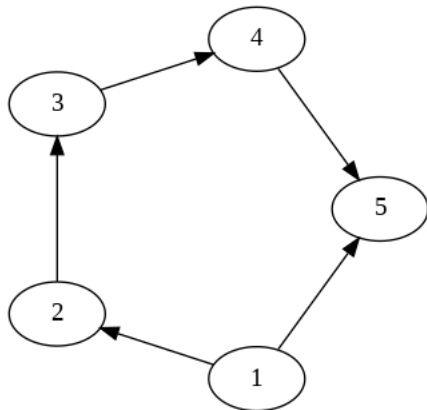


```
!apt install libgraphviz-dev
!pip install pygraphviz
import networkx as nx
from IPython.display import Image
```

Introduction

```
from networkx import nx_agraph, DiGraph

graph = DiGraph([(1, 2), (2, 3), (3, 4), (4, 5), (1, 5)])
nx_agraph.to_agraph(graph).draw('toy_graph.png', prog='circo')
Image('toy_graph.png')
```



Node Degree

```
from networkx import Graph

graph = Graph()
graph.add_nodes_from('ABCKLM')
graph.add_edges_from(['AC', 'BC', 'BK', 'KA', 'KC', 'LK'])

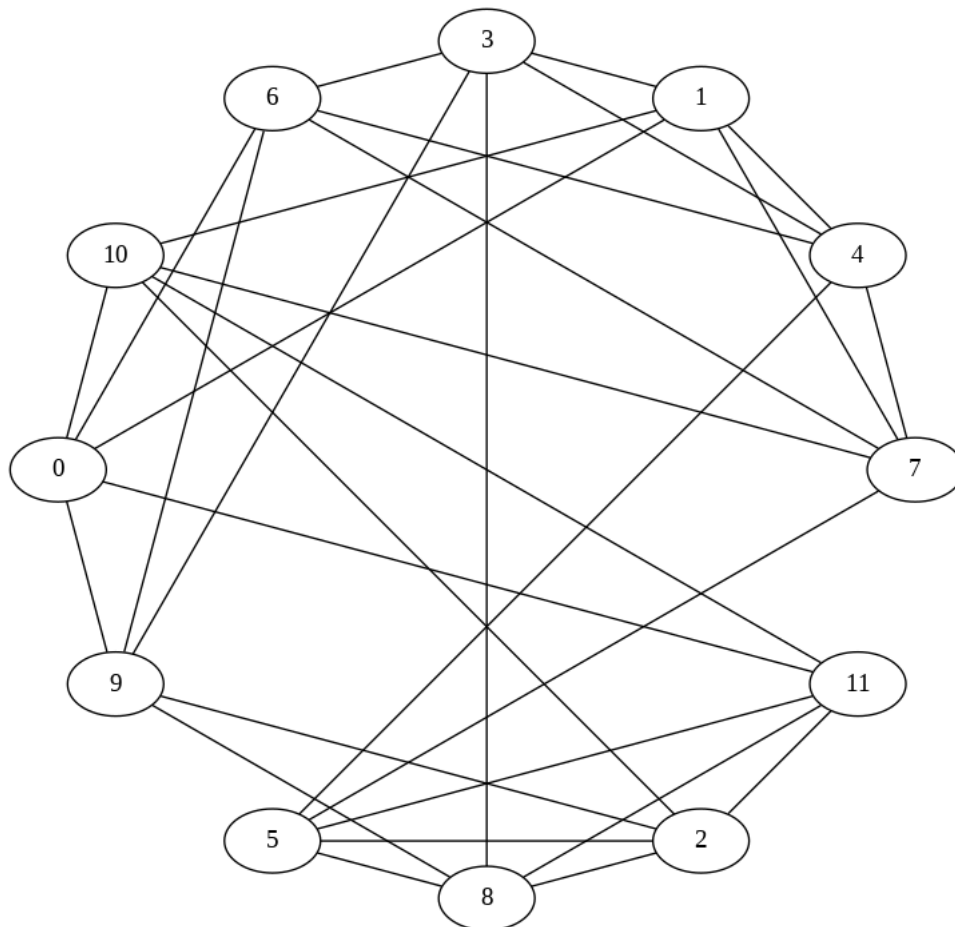
print(list(graph.degree()))
print(list(graph.neighbors('B')))
```

[('A', 2), ('B', 2), ('C', 3), ('K', 4), ('L', 1), ('M', 0)]
['C', 'K']

5-regular graph with 12 nodes:

```
from networkx import nx_agraph, random_regular_graph

graph = random_regular_graph(n=12, d=5)
nx_agraph.to_agraph(graph).draw('regular_graph.png', prog='circo')
Image('regular_graph.png')
```



```
from networkx import connected_components, Graph

edges = [(1, 2), (3, 8), (4, 6), (3, 6), (2, 5), (3, 7), (4, 8), (5, 1)]
print(list(connected_components(Graph(edges))))
```

```
[[{1, 2, 5}, {3, 4, 6, 7, 8}]]
```

```
from networkx import DiGraph

graph = DiGraph(['AB', 'BC', 'AD', 'BD', 'CD', 'DA'])

for node in ('A', 'D'):
    print(f'indeg({node})={graph.in_degree(node)}', end=' ')
    print(f'outdeg({node})={graph.out_degree(node)}', end=' ')

indeg(A)=1 outdeg(A)=2 indeg(D)=3 outdeg(D)=1
```

```
from networkx import Graph, shortest_path, shortest_path_length

g = Graph()
for u, v, w in [
    ('A', 'B', 3), ('A', 'F', 4), ('B', 'G', 1), ('F', 'G', 3),
    ('B', 'C', 4), ('C', 'G', 2), ('F', 'E', 8), ('G', 'E', 7),
    ('C', 'D', 5), ('E', 'D', 2), ('C', 'E', 1)
]:
    g.add_edge(u, v, weight=w)
```

```
print(shortest_path_length(g, source='A', target='D', weight='weight'))
print(shortest_path(g, source='A', target='D', weight='weight'))
```

```
9
['A', 'B', 'G', 'C', 'E', 'D']
```

Basic Graphs

Paths, Cycles, and Complete Graphs

```
from networkx import path_graph
```

```
graph = path_graph(n=5)
print(graph.nodes(), graph.edges())
```

```
[0, 1, 2, 3, 4] [(0, 1), (1, 2), (2, 3), (3, 4)]
```

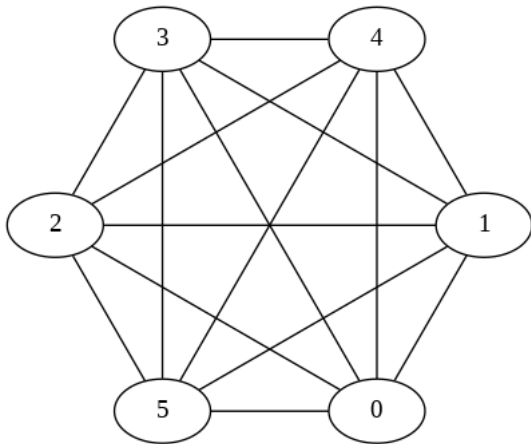
```
from networkx import cycle_graph
```

```
graph = cycle_graph(n=4)
print(graph.nodes(), graph.edges())
```

```
[0, 1, 2, 3] [(0, 1), (0, 3), (1, 2), (2, 3)]
```

```
from networkx import complete_graph, nx_agraph
```

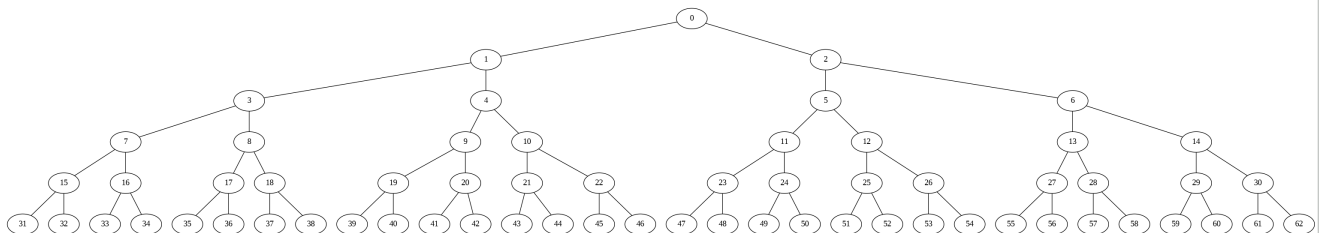
```
nx_agraph.to_agraph(complete_graph(n=6)).draw('k6.png', prog='circo')
Image('k6.png')
```



Trees

```
from networkx import balanced_tree, nx_agraph
```

```
nx_agraph.to_agraph(balanced_tree(2, 5)).draw('balanced_tree.png', prog='dot')
Image('balanced_tree.png')
```



Bipartite Graphs

```
from networkx import grid_2d_graph, is_bipartite  
  
print(is_bipartite(grid_2d_graph(7, 4)))
```

True

```
from networkx import complete_bipartite_graph, nx_agraph  
  
k47 = complete_bipartite_graph(4, 7)  
nx_agraph.to_agraph(k47).draw('k47.png', prog='dot')  
Image('k47.png')
```

